```typescript
// Basic function with type annotations
function greet(name: string): string {
  return `Hello, ${name}!`;
}

// Arrow function with explicit return type
const add = (a: number, b: number): number => {
  return a + b;
};

// Function with optional parameter
function greetOptional(name: string, greeting?: string): string {
  const g = greeting ?? "Hello";
  return `${g}, ${name}!`;
}

// Function with default parameter
function greetDefault(name: string, greeting = "Hi"): string {
  return `${greeting}, ${name}!`;
}

// Function with rest parameters
function sum(...numbers: number[]): number {
  return numbers.reduce((acc, n) => acc + n, 0);
}

// Generic function
function identity<T>(value: T): T {
  return value;
}

// Function type as a type alias
type MathOperation = (a: number, b: number) => number;

const multiply: MathOperation = (a, b) => a * b;
const divide: MathOperation = (a, b) => a / b;

// Higher-order function
function applyOperation(a: number, b: number, op: MathOperation): number {
  return op(a, b);
}

// Async function
async function fetchUser(id: number): Promise<{ id: number; name: string }> {
  // Simulated async operation
  return { id, name: `User ${id}` };
}

// Usage examples
console.log(greet("Alice"));                    // "Hello, Alice!"
console.log(add(5, 3));                         // 8
console.log(greetOptional("Bob"));              // "Hello, Bob!"
console.log(greetOptional("Bob", "Hey"));       // "Hey, Bob!"
console.log(greetDefault("Carol"));             // "Hi, Carol!"
console.log(sum(1, 2, 3, 4, 5));                // 15
console.log(identity<string>("test"));          // "test"
console.log(identity(42));                      // 42
console.log(applyOperation(10, 5, multiply));   // 50
console.log(applyOperation(10, 5, divide));     // 2

// Async usage
fetchUser(1).then(user => console.log(user));   // { id: 1, name: "User 1" }
```